

Benchmarking High-Performance Computing Strategies: A Comparative Analysis of Serial and Parallel N-Body Simulations in C, C++, and Python

Shirshendu Sekhar Mondal

December 11, 2025

Abstract

The gravitational N-body problem is a cornerstone of computational astrophysics, notoriously demanding due to its inherent computational complexity. This study performs a comparative performance analysis of the direct "all-pairs" N-body algorithm implemented across three programming languages: C, C++, and Python. We evaluate the impact of parallel computing using OpenMP for C/C++ and the Numba Just-In-Time (JIT) compiler for Python. Benchmarks conducted on an Apple Silicon (arm64) architecture reveal exponential performance degradation in pure interpreted Python. Conversely, C and C++ demonstrate near-identical baseline performance, with significant scaling achieved through OpenMP parallelization at larger particle counts (N). Notably, Python accelerated with Numba's parallel JIT compilation achieved execution speeds competitive with optimized parallel C++, demonstrating the viability of high-level languages for HPC tasks when appropriate tooling is utilized.

1 Introduction

In physics and astronomy, an N-body simulation predicts the individual motions of a group of celestial objects interacting with each other gravitationally. These simulations are widely used tools in astrophysics, ranging from investigating the dynamics of few-body systems like the Earth-Moon-Sun to understanding the large-scale structure of the universe.

The fundamental challenge of direct N-body simulations lies in their computational complexity. Because every body experiences a gravitational force from every other body in the system, a brute-force approach requires calculating interactions for all possible pairs, resulting in a time complexity of $O(N^2)$. As the number of bodies (N) increases, the computational cost grows quadratically, making serial execution on a single processor prohibitively slow for large systems.

To address this, high-performance computing relies heavily on parallelization—distributing the workload across multiple processing cores to speed up computation. The primary goal of this experiment is to quantify the performance differences between standard programming languages used in scientific computing and to evaluate the effectiveness of shared-memory parallelization techniques. We compare traditional compiled languages (C and C++) using the industry-standard OpenMP API against Python, a high-level language often criticized for its speed but increasingly utilized as a "glue" language in scientific workflows. We specifically examine if modern Python tools like Numba can bridge the performance gap with native C code.

2 Methodology

2.1 The Physical Model and Algorithm

The simulation relies on Newton’s Law of Universal Gravitation. For a system of N bodies, the total force $\vec{F}_i$ exerted on body i by all other bodies j is given by:

$$\vec{F}_i = \sum_{j=1, j \neq i}^N G \frac{m_i m_j}{|\vec{r}_j - \vec{r}_i|^3} (\vec{r}_j - \vec{r}_i) \quad (1)$$

Where G is the gravitational constant, m is mass, and $\vec{r}$ is the position vector. In our implementation, a softening factor ϵ^2 was added to the denominator ($|\vec{r}_{ij}|^2 + \epsilon^2$) to prevent numerical singularities when particles approach closer than a realistic physical radius.

We utilized the Direct ”All-Pairs” algorithm. For every time step, the program iterates through all N particles, and for each particle, it iterates through all remaining $N - 1$ particles to accumulate forces. This intrinsic double loop structure is the source of the $O(N^2)$ complexity but also presents a clear target for parallelization, as the force calculation for each particle is largely independent within a single time step.

2.2 Implementations and Parallelization Strategies

2.2.1 C and C++ (OpenMP)

The C and C++ implementations were designed to be as similar as possible to allow for direct comparison of language overhead. The C++ version utilizes ”std::vector” for dynamic memory management, testing the efficiency of C++ abstractions against raw C pointers.

For parallelization, we employed OpenMP (Open Multiprocessing), a standard shared-memory interface. By adding the compiler directive `#pragma omp parallel for schedule(static)` above the main outer loop, iterations are divided among available CPU threads. OpenMP was chosen because it utilizes the shared L3 cache between cores on the same processor, reducing memory usage compared to distributed memory systems like MPI.

2.2.2 Python (Pure vs. Numba)

Python’s standard interpreter (CPython) is known to be slow for tight numerical loops due to dynamic typing overhead and the Global Interpreter Lock (GIL). We tested a ”Pure Python” baseline using NumPy arrays for storage but standard Python ‘for’ loops for calculation.

To overcome Python’s performance limitations, we utilized Numba, an open-source JIT compiler that translates Python functions into optimized machine code using the LLVM compiler backend. By decorating the core physics function with `@jit(parallel=True)`, Numba automatically identifies parallelizable loops (similar to OpenMP) and compiles a multi-threaded native machine code version of the function.

2.3 Experimental Setup

The benchmarks were conducted on a macOS system with an Apple Silicon (arm64) processor.

- **Compilers:** GCC/G++ (Clang backend provided by Xcode and Homebrew) were used for C/C++, linking against the `libomp` library.
- **Python Environment:** Anaconda Python distribution with NumPy and Numba installed.

- **Parameters:** The simulation was run for a fixed number of 20 time steps. The number of bodies N was varied across $[100, 500, 1000, 2000]$ to observe scaling behavior. Time was measured using high-resolution system timers surrounding the core physics calculation loop.

3 Results and Analysis

The execution times for varying N were recorded and plotted on a logarithmic scale to accommodate the vast difference in performance between pure interpreted code and compiled parallel code. The results are presented in Figure 1.

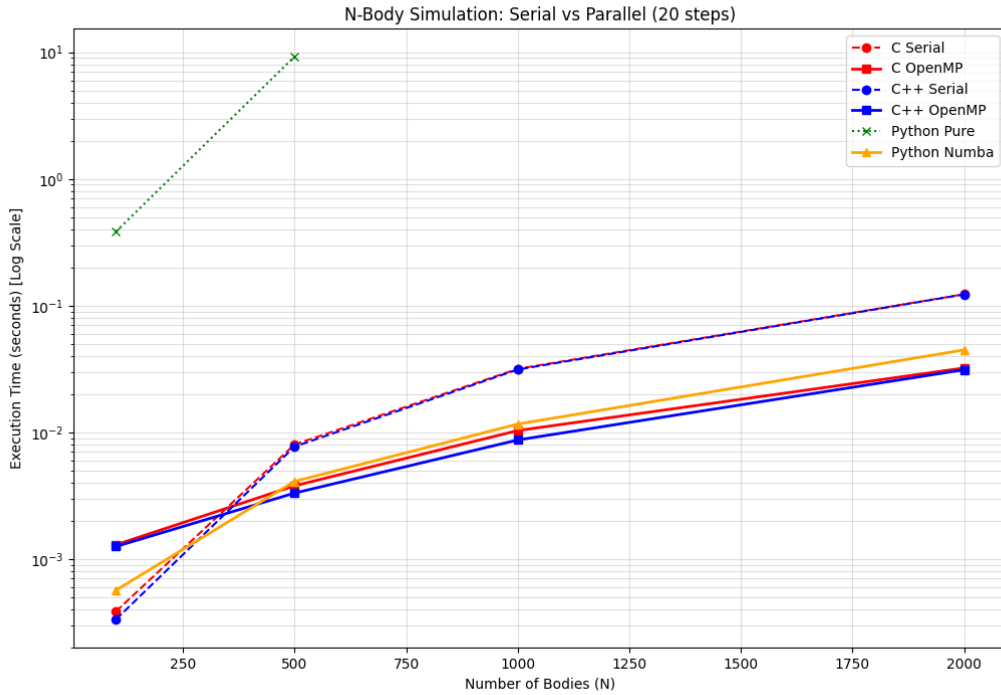


Figure 1: Performance comparison of N-Body simulation (20 steps) across varying particle counts (N). Note the logarithmic scale on the Y-axis representing execution time in seconds. Lower is better.

3.1 The Cost of Interpretation

The "Python Pure" line (green dotted, marked with 'x') demonstrates exponential performance degradation. At $N = 500$, the pure Python implementation is already orders of magnitude slower than the compiled languages. For $N = 1000$ and $N = 2000$, the execution time was so excessive that these tests were skipped by the benchmarking script. This confirms that pure Python loops are unsuitable for $O(N^2)$ numerical simulations.

3.2 C vs. C++ Baseline

The Serial C (red dashed circles) and Serial C++ (blue dashed circles) lines overlap almost perfectly throughout the test range. This indicates that modern C++ abstractions, such as

`std::vector` and struct-based object orientation, incur negligible zero-cost abstraction overhead compared to traditional C style array manipulation when compiled with optimization flags (-O3).

3.3 Parallel Scaling

The impact of parallelization becomes evident as N increases. At $N = 100$, the parallel OpenMP versions (solid lines) show little to no advantage over serial versions, and in some cases may be slightly slower due to the overhead cost of thread creation and management outweighing the computational workload.

However, as N grows to 1000 and 2000, the computational load dominates. The solid red and blue lines diverge significantly from their dashed serial counterparts, showing a substantial decrease in execution time. This validates the effectiveness of OpenMP for this embarrassing parallel problem structure.

3.4 Numba's Performance

The most significant result is the performance of "Python Numba" (orange solid line, triangles). It dramatically outperforms pure Python and serial C/C++. Remarkably, its performance trajectory closely follows the parallel C and C++ implementations. This demonstrates that Numba's JIT compiler successfully optimized the Python code and effectively utilized the underlying OpenMP backend to parallelize the loops. It proves that high-level syntax does not necessarily preclude high performance if the right compilation tools are used.

4 Conclusion

This experiment validated the theoretical computational challenges of N-body simulations and demonstrated the necessity of parallel computing for large-scale systems. While pure Python is insufficient for such tasks, the landscape of high-performance computing is evolving. We confirmed that C and C++ with OpenMP remain the gold standard for raw performance. However, the ability of Python, augmented with Numba JIT compilation, to achieve near-parity with optimized parallel C++ challenges the traditional notion that serious number-crunching requires abandoning high-level languages. For many scientific applications, the development speed of Python combined with the execution speed of Numba offers a highly compelling compromise.

A Source Code

A.1 C Implementation (nbody.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 // Conditional Include for OpenMP
6 #ifdef _OPENMP
7 #include <omp.h>
8 #else
9 #include <time.h>
10 #include <sys/time.h>
11 #endif
12
13 typedef struct {
14     float x, y, z;
15     float vx, vy, vz;
16     float mass;
17 } Body;
18
19 // Cross-platform timer function
20 double get_time() {
21     #ifdef _OPENMP
22         return omp_get_wtime();
23     #else
24         struct timeval tv;
25         gettimeofday(&tv, NULL);
26         return tv.tv_sec + tv.tv_usec * 1e-6;
27     #endif
28 }
29
30 void update_physics(Body* bodies, int N, float dt, int steps) {
31     for (int t = 0; t < steps; t++) {
32         // Only parallelize if OpenMP is active
33         #pragma omp parallel for schedule(static)
34         for (int i = 0; i < N; i++) {
35             float Fx = 0, Fy = 0, Fz = 0;
36             for (int j = 0; j < N; j++) {
37                 if (i == j) continue;
38                 float dx = bodies[j].x - bodies[i].x;
39                 float dy = bodies[j].y - bodies[i].y;
40                 float dz = bodies[j].z - bodies[i].z;
41                 float dist_sq = dx*dx + dy*dy + dz*dz + 1e-9f;
42                 float dist = sqrtf(dist_sq);
43                 float f = (1.0f * bodies[i].mass * bodies[j].mass) / (dist_sq *
44                     dist);
45                 Fx += f * dx;
46                 Fy += f * dy;
47                 Fz += f * dz;
48             }
49             bodies[i].vx += Fx * dt / bodies[i].mass;
50             bodies[i].vy += Fy * dt / bodies[i].mass;
51             bodies[i].vz += Fz * dt / bodies[i].mass;
52         }
53         for (int i = 0; i < N; i++) {
54             bodies[i].x += bodies[i].vx * dt;
55             bodies[i].y += bodies[i].vy * dt;
56             bodies[i].z += bodies[i].vz * dt;
57         }
58     }
```

```

59 }
60
61 int main(int argc, char** argv) {
62     if (argc != 3) { printf("Usage: %s <N> <steps>\n", argv[0]); return 1; }
63     int N = atoi(argv[1]);
64     int steps = atoi(argv[2]);
65     float dt = 0.01f;
66
67     Body* bodies = (Body*)malloc(N * sizeof(Body));
68
69     for (int i = 0; i < N; i++) {
70         bodies[i].x = (float)(rand() % 100);
71         bodies[i].y = (float)(rand() % 100);
72         bodies[i].z = (float)(rand() % 100);
73         bodies[i].vx = 0; bodies[i].vy = 0; bodies[i].vz = 0;
74         bodies[i].mass = 1.0f;
75     }
76
77     double start = get_time();
78     update_physics(bodies, N, dt, steps);
79     double end = get_time();
80
81     printf("%f", end - start);
82     free(bodies);
83     return 0;
84 }

```

Listing 1: C Implementation with OpenMP

A.2 C++ Implementation (nbody.cpp)

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <cstdlib>
5
6 #ifdef _OPENMP
7 #include <omp.h>
8 #else
9 #include <ctime>
10 #include <sys/time.h>
11 #endif
12
13 struct Body {
14     float x, y, z;
15     float vx, vy, vz;
16     float mass;
17 };
18
19 double get_time() {
20     #ifdef _OPENMP
21         return omp_get_wtime();
22     #else
23         struct timeval tv;
24         gettimeofday(&tv, NULL);
25         return tv.tv_sec + tv.tv_usec * 1e-6;
26     #endif
27 }
28
29 void update_physics(std::vector<Body>& bodies, float dt, int steps) {
30     int N = bodies.size();
31     for (int t = 0; t < steps; t++) {
32         #pragma omp parallel for schedule(static)
33         for (int i = 0; i < N; i++) {
34             float Fx = 0, Fy = 0, Fz = 0;
35             for (int j = 0; j < N; j++) {
36                 if (i == j) continue;
37                 float dx = bodies[j].x - bodies[i].x;
38                 float dy = bodies[j].y - bodies[i].y;
39                 float dz = bodies[j].z - bodies[i].z;
40                 float dist_sq = dx*dx + dy*dy + dz*dz + 1e-9f;
41                 float dist = std::sqrt(dist_sq);
42                 float f = (1.0f * bodies[i].mass * bodies[j].mass) / (dist_sq *
43                     dist);
44                 Fx += f * dx;
45                 Fy += f * dy;
46                 Fz += f * dz;
47             }
48             bodies[i].vx += Fx * dt / bodies[i].mass;
49             bodies[i].vy += Fy * dt / bodies[i].mass;
50             bodies[i].vz += Fz * dt / bodies[i].mass;
51         }
52         for (int i = 0; i < N; i++) {
53             bodies[i].x += bodies[i].vx * dt;
54             bodies[i].y += bodies[i].vy * dt;
55             bodies[i].z += bodies[i].vz * dt;
56         }
57     }
58 }
59
60 int main(int argc, char** argv) {
```

```

61     if (argc != 3) return 1;
62     int N = std::atoi(argv[1]);
63     int steps = std::atoi(argv[2]);
64     float dt = 0.01f;
65
66     std::vector<Body> bodies(N);
67     for (int i = 0; i < N; i++) {
68         bodies[i] = {
69             (float)(rand() % 100), (float)(rand() % 100), (float)(rand() % 100)
70             ,
71             0, 0, 0, 1.0f
72         };
73     }
74
75     double start = get_time();
76     update_physics(bodies, dt, steps);
77     double end = get_time();
78
79     std::cout << (end - start);
80     return 0;

```

Listing 2: C++ Implementation with OpenMP

A.3 Python Implementation (nbody.py)

```
1 import numpy as np
2 import time
3 import sys
4 from numba import njit, prange
5
6 # --- 1. Pure Python (Very Slow, for baseline) ---
7 def update_physics_pure(pos, vel, mass, dt, steps):
8     N = len(pos)
9     for _ in range(steps):
10         for i in range(N):
11             Fx, Fy, Fz = 0.0, 0.0, 0.0
12             for j in range(N):
13                 if i == j: continue
14                 dx = pos[j,0] - pos[i,0]
15                 dy = pos[j,1] - pos[i,1]
16                 dz = pos[j,2] - pos[i,2]
17                 dist_sq = dx*dx + dy*dy + dz*dz + 1e-9
18                 dist = dist_sq ** 0.5
19                 f = (mass[i] * mass[j]) / (dist_sq * dist)
20                 Fx += f * dx
21                 Fy += f * dy
22                 Fz += f * dz
23             vel[i,0] += Fx * dt / mass[i]
24             vel[i,1] += Fy * dt / mass[i]
25             vel[i,2] += Fz * dt / mass[i]
26         pos += vel * dt
27
28 # --- 2. Numba Parallel (Uses OpenMP backend) ---
29 @njit(parallel=True, fastmath=True)
30 def update_physics_numba(pos, vel, mass, dt, steps):
31     N = len(pos)
32     for _ in range(steps):
33         # prange explicitly tells Numba to parallelize this loop (like #pragma
34         # omp parallel for)
35         for i in prange(N):
36             Fx, Fy, Fz = 0.0, 0.0, 0.0
37             for j in range(N):
38                 if i == j: continue
39                 dx = pos[j,0] - pos[i,0]
40                 dy = pos[j,1] - pos[i,1]
41                 dz = pos[j,2] - pos[i,2]
42                 dist_sq = dx*dx + dy*dy + dz*dz + 1e-9
43                 dist = dist_sq ** 0.5
44                 f = (mass[i] * mass[j]) / (dist_sq * dist)
45                 Fx += f * dx
46                 Fy += f * dy
47                 Fz += f * dz
48             vel[i,0] += Fx * dt / mass[i]
49             vel[i,1] += Fy * dt / mass[i]
50             vel[i,2] += Fz * dt / mass[i]
51
52         # Parallelize position update too
53         for i in prange(N):
54             pos[i,0] += vel[i,0] * dt
55             pos[i,1] += vel[i,1] * dt
56             pos[i,2] += vel[i,2] * dt
57
58 if __name__ == "__main__":
59     mode = sys.argv[1] # "pure" or "numba"
60     N = int(sys.argv[2])
61     steps = int(sys.argv[3])
```

```

61
62 pos = np.random.rand(N, 3).astype(np.float32)
63 vel = np.zeros((N, 3), dtype=np.float32)
64 mass = np.ones(N, dtype=np.float32)
65
66 start = time.time()
67 if mode == "pure":
68     update_physics_pure(pos, vel, mass, 0.01, steps)
69 elif mode == "numba":
70     # First call includes compilation time, so we run a dummy first
71     update_physics_numba(pos, vel, mass, 0.01, 1)
72     start = time.time() # Reset timer
73     update_physics_numba(pos, vel, mass, 0.01, steps)
74
75 print(time.time() - start)

```

Listing 3: Python Implementation using Numba JIT

A.4 Benchmark and comparison (benchmark_{andplot}.py)

```
1 import subprocess
2 import matplotlib.pyplot as plt
3 import sys
4 import os
5
6 # --- Configuration ---
7 body_counts = [100, 500, 1000, 2000]
8 steps = 20
9
10 # Executables
11 executables = {
12     "C Serial": "./nbody_c_serial",
13     "C OpenMP": "./nbody_c_omp",
14     "C++ Serial": "./nbody_cpp_serial",
15     "C++ OpenMP": "./nbody_cpp_omp"
16 }
17
18 results = {
19     "C Serial": [], "C OpenMP": [],
20     "C++ Serial": [], "C++ OpenMP": [],
21     "Python Pure": [], "Python Numba": []
22 }
23
24 def run_command(cmd, label, N):
25     try:
26         # Run command and capture output
27         output = subprocess.check_output(cmd, shell=True, stderr=subprocess.
28             STDOUT)
29         time_taken = float(output.strip())
30         return time_taken
31     except subprocess.CalledProcessError as e:
32         print(f"\nError running {label} with N={N}:")
33         print(e.output.decode())
34         return None
35     except ValueError:
36         return None
37
38 if __name__ == "__main__":
39     print(f"{'='*60}")
40     print(f"Starting N-Body Benchmark")
41     print(f"Python Interpreter: {sys.executable}") # This confirms which python
42     print(f"{'='*60}\n")
43
44     print(f"{'Type':<15} | {'N':<6} | {'Time (s)':<10}")
45     print("-" * 35)
46
47     for N in body_counts:
48         # 1. Run C/C++ Executables
49         for label, exe_path in executables.items():
50             if os.path.exists(exe_path):
51                 cmd = f"{exe_path} {N} {steps}"
52                 t = run_command(cmd, label, N)
53                 results[label].append(t)
54                 if t: print(f"{label:<15} | {N:<6} | {t:.5f}")
55             else:
56                 results[label].append(None)
57
58     # 2. Run Python (Pure)
59     # Note: We use 'nbody.py' here because your logs show that is your
60     filename
```

```

59     if N <= 500:
60         # sys.executable forces the script to use the CURRENT python (
Anaconda)
61         cmd = f"{sys.executable} nbody.py pure {N} {steps}"
62         t = run_command(cmd, "Python Pure", N)
63         results["Python Pure"].append(t)
64         if t: print(f"'Python Pure':<15} | {N:<6} | {t:.5f}")
65     else:
66         results["Python Pure"].append(None)
67         print(f"'Python Pure':<15} | {N:<6} | SKIPPED")
68
69     # 3. Run Python (Numba)
70     cmd = f"{sys.executable} nbody.py numba {N} {steps}"
71     t = run_command(cmd, "Python Numba", N)
72     results["Python Numba"].append(t)
73     if t: print(f"'Python Numba':<15} | {N:<6} | {t:.5f}")
74
75     print("-" * 35)
76
77     # --- Plotting ---
78     print("\nGenerating plot...")
79     plt.figure(figsize=(12, 8))
80
81     styles = {
82         "C Serial":      {'color': 'red', 'linestyle': '--', 'marker': 'o'},
83         "C OpenMP":      {'color': 'red', 'linestyle': '-', 'marker': 's', '
linewidth': 2},
84         "C++ Serial":    {'color': 'blue', 'linestyle': '--', 'marker': 'o'},
85         "C++ OpenMP":    {'color': 'blue', 'linestyle': '-', 'marker': 's', '
linewidth': 2},
86         "Python Pure":   {'color': 'green', 'linestyle': ':', 'marker': 'x'},
87         "Python Numba":  {'color': 'orange', 'linestyle': '-', 'marker': '^', '
linewidth': 2}
88     }
89
90     for label, times in results.items():
91         valid_data = [(n, t) for n, t in zip(body_counts, times) if t is not
None]
92         if valid_data:
93             x, y = zip(*valid_data)
94             plt.plot(x, y, label=label, **styles.get(label, {}))
95
96     plt.xlabel('Number of Bodies (N)')
97     plt.ylabel('Execution Time (seconds) [Log Scale]')
98     plt.title(f'N-Body Simulation: Serial vs Parallel ({steps} steps)')
99     plt.legend()
100    plt.grid(True, which="both", ls="-", alpha=0.4)
101    plt.yscale('log')
102
103    plt.savefig('nbody_benchmark_results.png')
104    print(f"Done! Plot saved to 'nbody_benchmark_results.png'")

```

Listing 4: Benchmark and comparison using python